- The script recommender will consume the data product through a REST API.
- Data analysts want to consume it using SQL statements to help in salary negotiations.
- We discover the following quality attributes:
    - *Privacy*—Only the production team and analyst can have access to cast personal data.
    - *Security*—The payrolls of a cast include sensitive data. A data breach would lead us to million-dollar lawsuits from movie stars.
    - *Compliance*—We are keeping sensitive personal information. This is why GDPR and other privacy regulations apply here.
- The script recommender and other potential consumers are not real-time applications. They will be consuming data no more than once a day. Scalability, availability, and performance are not problems.
- Data analysts will perform their analysis a few times a year.

In this Cast data product example, we deal with a prevalent problem: how to expose data from monolithic, usually legacy, applications. Before we jump into the solution, we want to explain an interesting pattern called *turning the database inside out*.

TURNING THE DATABASE INSIDE OUT

Turning the database inside out is a pattern introduced by Martin Kleppmann (http://mng.bz/z4WB). In this pattern, Kleppmann combines four ideas:

- A database replication mechanism, whereby databases are replicated using asynchronous events transferred between the leader and a follower
- Secondary indexes
- Caching
- Materialized views

All of these are derived data, exactly like a data product. His premise is to create a magically self-updating cache. The way to do that is to expose a